Highlight key:
“Red = delete”
“Yellow = change”
“Blue = check term coinage”
“Green = i don’t know what this, explain the term and discuss further”
“Purple = what is the purpose of this sentence”
“Orange = my change i made”
“Pink = check if it is a claim or fact”

Go ahead and reduce number of hyphenated terms strictly to only ones that are mentioned in the original Fortran code, or make the case for why to keep it.
No EM dashes

Graph Neural Network Surrogates for a Coupled Heat-Conduction and Pyrolysis Solver in Thermal Protection System Design

Grey Goodwin, University of Kentucky

Abstract

Thermal protection system (TPS) design relies on repeated evaluation of an expensive, tightly coupled physics solver that resolves heat conduction, pyrolysis, pyrolysis-gas transport, and aerothermal surface chemistry. The cost of those evaluations bottlenecks design-space exploration. We present early work toward a graph neural network (GNN) surrogate that learns the per-step evolution of a 1D heat-shield pyrolysis solver, with the goal of running design loops far faster than the reference code. As a foundation, we re-implement a trusted Fortran reference solver in modular, vectorized Python and verify it against the original by column-level comparison of its thermocouple and density outputs. The port reproduces the reference to machine precision on the conduction-plus-pyrolysis case and to 0.064 K on the full gas-plus-aerothermal case. Using data generated by the validated solver, we then train a per-cell message-passing GNN that reproduces a held-out heating scenario over a full 60 s rollout to a mean error of 7.6 K, roughly half a percent to one percent of the temperature field, and tracks the pyrolysis front to within a fraction of a millimeter. This work extends an existing GNN-surrogate program from canonical flux problems to a multi-physics engineering solver.

---

1. Introduction

Thermal protection systems shield a vehicle from the intense aerothermal heating of atmospheric entry. Designing one means choosing a material and a thickness that protects the structure beneath the heat shield from temperature without adding excessive mass. The physics that governs this is a coupled, strongly nonlinear problem: heat conducts into the material, the material pyrolyzes (chemically decomposes) as it heats, the pyrolysis gas percolates out through the charring solid, and the exposed surface exchanges energy with the hot boundary layer.

A high-fidelity solver that captures this coupling is expensive to run, and TPS design needs many such runs: parameter sweeps, sizing studies, and optimization loops each call the solver hundreds or thousands of times. That cost is the bottleneck this work targets. We propose a machine-learned surrogate trained on data from a trusted reference solver, so that design loops can replace most solver calls with fast surrogate evaluations.

We use a graph neural network for the surrogate because the discretized solver is naturally a graph: the mesh cells are nodes, and adjacent cells are connected by edges. A GNN that shares its weights across all cells is mesh-native and, in principle, applies to a mesh of any size, which is the property we ultimately want for a tool that must run at varying resolutions.

This paper reports two things. First, a faithful Python re-implementation of the reference Fortran solver, verified against the original to machine precision on the simpler test case and to a small fraction of a kelvin on the full multi-physics case. Second, a first working surrogate trained on data from that validated solver, accurate over a full rollout on a held-out scenario. We are explicit about scope: the surrogate is demonstrated here on the conduction-plus-pyrolysis case, and extending it to the gas-plus-aerothermal case and to cross-resolution operation are stated as future work, not claimed results.

2. Reference solver

The reference is a 1D finite-volume heat-shield code written in Fortran (the author's `1Dheat.f` and its associated include files). It models, in one spatial dimension through the thickness of the slab:

- **Heat conduction** with temperature-dependent and char-state-dependent material properties.
- **Pyrolysis**, the heat-driven decomposition of the solid, modeled as a set of Arrhenius reactions that consume reactive species and release gas.
- **Pyrolysis-gas transport** through the porous char (Darcy flow plus advection), which carries energy and couples back into the temperature field.

- **An aerothermal surface boundary**, including a B-prime surface-chemistry table, a blowing correction, and a wall-enthalpy balance, representing the interaction with the hot boundary layer.

The governing energy equation is the conduction-diffusion balance

$$\rho\, c_p\, \frac{\partial T}{\partial t} = \frac{\partial}{\partial x}\!\left(k\, \frac{\partial T}{\partial x}\right) + (\text{pyrolysis and gas source terms}),$$

discretized on a finite-volume mesh. Each cell carries a temperature and a set of species densities; the conduction term is evaluated from temperature differences across cell faces, and the pyrolysis term advances each species by a closed-form Arrhenius update. Material properties are blended between virgin and char states by a char-progress variable derived from the local density.

Three reference cases ship with the code and are used throughout: `aw1`, a conduction-plus-pyrolysis case with a prescribed wall-temperature history and the gas physics off, and `aw2_tc21` and `aw2_tc22`, which add the full gas and aerothermal boundary. (As shipped, `aw2_tc22` is configured identically to `aw2_tc21`; see Section 4.1.)

3. Methodology

3.1 Python port

The solver was re-implemented in Python one physics module at a time: grid and geometry setup, mixture properties (conductivity, heat capacity, enthalpy), the Arrhenius pyrolysis update, the gas solve, and the aerothermal boundary. Where a coding choice affected the numerical result, the Python deliberately matched the Fortran. The thermocouple sampling in particular was rewritten to mirror the reference probe-sampling routine, clamping to the face value at domain boundaries and reconstructing the face value by interpolation in the interior.

Several port details were matched to specific behavior in the Fortran and are worth recording, since each one materially affected agreement:

- Thermocouple positions are specified as depth from the right wall, so the internal coordinate is the slab length minus that depth.
- The face density used to evaluate the face conductivity is the pre-pyrolysis density, matching the order in which the Fortran updates state.

- Output is written before the physics step, matching the reference time-offset convention.

3.2 Verification approach

Verification is done by direct column-by-column comparison against the Fortran output file from a run with identical configuration. For every output column (time, each thermocouple temperature, and each density), we report the maximum absolute error, the mean absolute error, and the maximum relative error across the run. This is a strict test: it compares the full time history at every recorded probe, not just a summary statistic.

3.3 Training-data generation

Once verified, the Python solver generates the training data. Each run records the full per-cell field over time (temperature, bulk density, the per-species densities, and, in gas runs, the gas pressure and mass flux), including the two ghost cells at the domain ends that carry the boundary forcing. A single recorded trajectory can later be turned into training pairs at any time-step spacing without re-running the solver. The first dataset varies the boundary forcing, a sweep of wall-temperature histories, on the fast gas-off `aw1` case.

3.4 GNN surrogate architecture

The surrogate is a per-cell message-passing network. Each finite-volume cell is a node carrying the state vector `[T, ρ, $\rho_i$, porosity]`. Adjacent cells are joined by edges whose features are relative: the difference between the neighbor's state and the cell's own state, together with the cell spacing. This relative encoding mirrors the physics, since the conduction flux between two cells depends on their temperature difference and their spacing.

A forward step encodes the node and edge features with small multilayer perceptrons, performs one round of message passing (each cell aggregates the encoded messages from its left and right neighbors and updates its own hidden state), and decodes a predicted change in the cell's state. The change is added to the current state and the model is rolled forward in time, exactly as the solver advances its own update. A single message-passing round is used, matching the nearest-neighbor reach of the conduction stencil. Because the weights are shared across all cells, the same trained model applies to a mesh of any length.

The model predicts only the independent degrees of freedom, temperature and the per-species densities; the bulk density and porosity are derived from the predicted species densities. Training minimizes the error of a short multi-step rollout rather than a single step, which is what controls the accumulation of error over a long rollout (see Section 4.2).

4. Results

4.1 Port validation

aw1 (conduction plus pyrolysis): bit-exact. All fifteen output columns match the Fortran output to approximately 1e-8, machine precision, including every interior thermocouple. The conduction and pyrolysis port is exact.

aw2_tc21 (full gas plus aerothermal): to 0.064 K. With the gas physics enabled, compared against the same-configuration Fortran output:

Quantity	Agreement
All temperature columns	max 0.064 K, mean about 4e-4 K
Hot aerothermal surface	0.06 K
Densities	about 1e-3 to 1e-9
Time, cold back face	bit-exact

During verification, an apparent early interior drift and a roughly 37.7 K back-face offset both proved to be artifacts of how the thermocouples were being sampled, not numerical error; both vanished once the probe sampling matched the reference routine. This is a useful caution: comparing two temperature fields through a sampling layer can manufacture a discrepancy that is not present in the physics itself.

aw2_tc22. As shipped, this case is byte-for-byte identical in configuration to `aw2_tc21` (the same case file, thermocouple locations, and flux history, and the case-name field still reads `aw2_tc21`), so it does not add an independent validation condition. The Python run reproduces it identically. Confirmation of whether `tc22` was intended to carry a distinct configuration would be valuable.

4.2 Surrogate accuracy

Trained on solver-generated trajectories, the surrogate is rolled forward on its

own predictions over a held-out `aw1` heating scenario (gas off) for the full 60 s, with the known boundary forcing re-imposed at each step:

- **Mean rollout error of 7.6 K**, roughly half a percent to one percent of the 298 to 1500 K temperature field.
- The surrogate **tracks the char and pyrolysis front to within a fraction of a millimeter**.

Reaching this level required training on multi-step rollouts rather than single steps. A single-step-trained model is accurate for one prediction but accumulates error and drifts once it is rolled forward freely; the end-of-trajectory error fell from several hundred kelvin to tens of kelvin when training was switched to a multi-step rollout loss. This error-accumulation behavior, and the multi-step remedy, are consistent with prior surrogate work in this program.

4.3 Wall time

The Fortran solver runs the `aw2_tc21` case in about 3 minutes; the present unoptimized Python implementation takes about 15.6 minutes, roughly five times slower, as expected for interpreted, array-based code matched against compiled Fortran. The Python port's role is to generate verified training data and to be a readable reference, not to be fast; the speed argument of this program is the surrogate, whose wall-time advantage over the solver is the subject of ongoing measurement.

5. Discussion

The results establish two things. First, the multi-physics reference solver can be reproduced faithfully in a modular, readable Python code: to machine precision on the conduction-plus-pyrolysis case, and to a small fraction of a kelvin on the full gas-plus-aerothermal case. That fidelity is what makes the solver usable as a data generator, since a surrogate is only as trustworthy as the data it learns from.

Second, a per-cell message-passing GNN trained on that data is an accurate surrogate for the conduction-plus-pyrolysis case over a full rollout, including faithful tracking of the moving pyrolysis front. The architecture's relative edge features give it the same local information the physics uses, namely the neighbor temperature difference and the cell spacing, and its weight sharing across cells is what makes it mesh-native.

The main limitations are clear and define the path forward. The surrogate is demonstrated on the gas-off case and on a single mesh resolution. The verification caution noted in Section 4.1, that a sampling layer can mask or invent discrepancies, applies equally when comparing a surrogate field against a solver field and is worth carrying into the surrogate evaluation.

6. Future work

- **Gas and aerothermal coverage.** Extend the surrogate to the `aw2` gas-plus-aerothermal case, which adds gas pressure and mass flux to the per-cell state.
- **Mesh-resolution generalization.** Train across a range of mesh resolutions so that a single model runs at resolutions it did not see in training, exploiting the weight sharing the architecture already provides.
- **Speedup measurement.** Quantify the surrogate's wall-time advantage over the solver across a design-relevant set of evaluations.
- **Physics extensions.** Add radiation and surface recession, and extend the mesh to two and three dimensions.

Acknowledgments

This work builds on the reference heat-shield solver developed by Dr. Martin, and on an existing graph-neural-network surrogate program within the group.